

Audio Deepfake Detection for Real-World Call (VoIP) Conditions

Raushan Vivek (CS25M115), IIT Tirupati (M.Tech, CSE)

GitHub link:- https://github.com/raushanvivek9/Deepfake_VOIP

Approach Overview

The detector is an ECAPA-TDNN backbone (SpeechBrain implementation) trained as a binary classifier (0 = real/bonafide, 1 = fake/spoof) on 80-bin log-mel spectrograms extracted from 4-second, 16 kHz audio clips. The pipeline is: clean and balance the provided fake set and LibriSpeech, split speaker-disjointly into train/val/test, train with on-the-fly VoIP-style degradation as augmentation, and evaluate on both a clean and a VoIP-degraded version of the same held-out test clips. Model selection (checkpointing and early stopping) is driven entirely by validation EER under VoIP degradation, not clean-set EER — this distinction turned out to matter a great deal.

Datasets Used

As required by the brief, every dataset used is listed below with its source. No additional open-source fake/spoof datasets (e.g. ASVspoof, WaveFake) were incorporated in this submission.

- **Fake / spoof (provided):** the ~40-hour synthesized-speech set supplied for the assignment.
- **Real / bonafide:** LibriSpeech *train-clean-100* (<https://www.openslr.org/resources/12/>), downloaded directly. 28,539 raw FLAC files.
- **Noise (for VoIP augmentation, not a speech dataset):** MUSAN (<https://www.openslr.org/resources/17/>) — only the *noise/* subset (930 files) was extracted; the *music/* and *speech/* subsets were not used.

Data Cleaning

Every file went through the same chain: decode and resample to 16 kHz mono → reject if corrupt/unreadable → trim leading/trailing silence (30 dB threshold) → reject if the trimmed clip is under 1.0s (near-total silence) → cap clips longer than 15.0s at 15.0s (keeps training clips a manageable, comparable length) → reject if WebRTC VAD measures speech in under 30% of 30ms frames (filters non-speech / mostly-noise clips).

Source	Raw files	Kept	Rejected (corrupt/silent)	Rejected speech	(non-)	Duration cleaning	after
Provided fake	11,985	11,939	46	0		17.77 h	
LibriSpeech (real)	28,539	28,538	1	0		96.20 h	

Note the fake set's cleaned total (17.77h) is well under the ~40h raw figure. This is expected, not a bug: the 15-second cap truncates (rather than discards) long clips, and it appears a meaningful share of the provided fake files were considerably longer than 15 seconds. The reported hours reflect what the model actually trains/evaluates on.

Balancing Real vs. Fake

Classes were balanced by total duration, not file count, since average clip length differs substantially between the two sources. Target = min(fake hours, real hours) = 17.77h. The fake set (already the smaller class) was kept in full. The real set was downsampled at the speaker level — shuffling whole speakers into or out of the cut, so a speaker's clips are never split across the boundary — down to 17.88h.

Class	Files kept	Duration
Fake	11,939	17.77 h
Real	5,273	17.88 h

One consequence worth flagging honestly: equal total hours does not mean equal file counts — the balanced set has about $2.3\times$ more fake clips than real clips, because average fake clip duration is shorter. Since the loss is computed per-clip (not per-hour), the fake class receives more individual gradient updates per epoch than the real class even though the two classes are duration-balanced. This was not corrected for in this submission (see Section 9).

Speaker-Disjoint Train/Val/Test Split

Speakers (not individual clips) were shuffled and assigned 70% / 15% / 15% to train / val / test. For real audio this uses the true LibriSpeech speaker ID; for fake audio each file is treated as its own “speaker” (prefixed fake_<filename> to avoid accidental collision with real speaker IDs), so the split is file-disjoint on the fake side. An automated assertion confirms no speaker_id or filepath appears in more than one split before the manifest is saved.

Split	Real (label 0)	Fake (label 1)	Total
Train	3,548	8,343	11,891
Val	773	1,782	2,555
Test	952	1,814	2,766

Features and Model

- **Features:** 80-bin log-mel spectrogram (n_fft=400, win_length=400 [25ms], hop_length=160 [10ms]) computed on 4-second clips (randomly cropped if longer, zero-padded if shorter).
- **Backbone:** ECAPA-TDNN (SpeechBrain) — channels [512,512,512,512,1536], kernel sizes [5,3,3,3,1], dilations [1,2,3,4,1], attention_channels=128, embedding_dim=192.
- **Head:** Linear(192→128) → ReLU → Dropout(0.3) → Linear(128→2), trained end-to-end (not a frozen-embedding classifier).
- **Size:** 6.22M parameters total.

Simulating VoIP Conditions

This is the part of the brief we treated as the actual heart of the task. The degradation pipeline, applied on-the-fly (a different random draw every time a clip is sampled):

- **Codec round-trip** — randomly Opus (24kbps, via ffmpeg libopus encode/decode) or G.711 μ -law (8kHz, via ffmpeg pcm_mulaw encode/decode).
- **Bandwidth** — downsample to 8kHz and back up to 16kHz, simulating telephone-narrowband channels regardless of which codec was chosen.
- **Noise** — mix in a randomly chosen MUSAN noise clip at a random SNR from {0, 5, 10, 15} dB.

Packet loss and jitter simulation (listed as optional in the brief) were not implemented in this submission — see Section 9.

Training: each clip is degraded with probability 0.5 per training step, so over many epochs the model sees every clip in both clean and degraded form. **Evaluation:** two loaders per split — clean (no degradation) and voip (degradation probability forced to 1.0) — built from the exact same underlying clips, so the clean-vs-VoIP gap reported below isolates the effect of degradation rather than any difference in which clips were sampled.

Effect on held-out test-set accuracy:

Condition	EER	AUC	Accuracy	F1
Clean	0.0296	0.9965	0.9729	0.9793
VoIP-degraded (codec + noise)	0.1482	0.9251	0.8568	0.8879

VoIP degradation costs roughly 11.9 points of EER (2.96% → 14.82%) and 11.6 points of accuracy on the same held-out speaker-disjoint test clips. This gap is reported honestly, in line with the brief's core message: clean-audio performance is a poor proxy for real-call performance, and a model has to be evaluated (and selected) under degradation to mean anything.

What We Tried, and What Failed (and Why)

The single most useful thing this run surfaced was a direct demonstration of the failure mode the brief warns about. The per-epoch validation log below tracks both the clean-set EER and the VoIP-degraded EER side by side:

Epoch	Train loss	Val clean EER	Val VoIP EER	Checkpoint
1	0.3412	0.1800	0.2960	best
2	0.2987	0.1200	0.2710	best
3	0.2409	0.1109	0.2199	best
4	0.2175	0.1435	0.2425	
5	0.1994	0.1629	0.2816	
6	0.1904	0.0743	0.2392	
7	0.1856	0.1256	0.2085	best
8	0.1745	0.0622	0.1992	best
9	0.1655	0.0902	0.2241	
10	0.1674	0.0610	0.2094	
11	0.1541	0.0724	0.2404	
12	0.1452	0.0931	0.2082	
13	0.1362	0.0691	0.2195	
14	0.1290	0.1077	0.2386	
15	0.1190	0.0463	0.2119	
16	0.1173	0.0517	0.2206	early stop

Val-clean EER kept falling through epoch 15 (down to 4.6%), which would look like the model was still improving if clean accuracy were the only thing being watched. But val-VoIP EER peaked (best) at epoch 8 (19.9%) and never improved again in the following 8 epochs, triggering early stopping at epoch 16. In other words: past epoch 8, additional training kept sharpening cues that only help on clean audio — and made no improvement, or actively lost ground, on the metric that determines whether the model is any good on a real call.

The fix we adopted: checkpoint selection and early-stopping patience were both driven by val-VoIP EER, never val-clean EER. Had we selected on clean EER instead, we would have kept training to epoch 15 and shipped a checkpoint that looked better on paper (4.6% clean EER) but was demonstrably worse under the actual conditions the brief cares about.

Within the two-week/single-notebook budget, this was run as one architecture end-to-end (ECAPA-TDNN) rather than a wider architecture or augmentation-strength search; the next iterations we'd want to run are listed in Section 9.

What We'd Do With More Time

- Add explicit packet-loss and jitter simulation to the degradation pipeline (listed as optional in the brief; currently only codec, bandwidth, and additive noise are modeled).
- Add G.729 / AMR-NB codec paths alongside Opus/G.711 for broader real-world call coverage.
- Correct for the ~2.3:1 fake:real file-count imbalance left over from duration-based balancing (e.g. a class-weighted loss or per-class sampling), and measure whether it was actually costing anything — its effect was never isolated in this run.
- Pull in more real-speech diversity (Mozilla Common Voice, VCTK) and consider an additional open fake/spoof set (ASVspoof, WaveFake) to test generalization beyond LibriSpeech-style read speech and the specific TTS engines in the provided set.
- Per-source and per-condition error breakdown — which codec, SNR level, or TTS engine drives the most VoIP-condition errors — including listening to misclassified clips directly, as suggested in the brief.
- Report metrics at the val-set EER-operating threshold rather than the flat 0.5 cutoff used here, for a stricter, decision-calibrated comparison (flagged as a to-do in results.md).
- Scale up training: a DistributedDataParallel version of this same pipeline targeting a multi-GPU node already exists, which would allow a real architecture/augmentation sweep instead of a single run.

Reproducibility

See result.md for the exact, numbered steps to reproduce this result end-to-end, including how to obtain each dataset.